

Explorando a Memória Cache: Uma Visão Abrangente

Nome dos autores: Gabriel Tomazzoni Mazzarotto, Reginaldo Devens Araldi

Resumo

O projeto de computadores enfrenta o desafio histórico de fornecer operandos ao processador na velocidade necessária. A solução inclui o uso crucial da Memória Cache para reduzir a latência de acesso e aumentar a taxa de acertos, melhorando o desempenho do sistema. Este estudo foca na simulação de uma Memória Cache associativa por conjunto com arquitetura configurável. Analisa-se o impacto do tamanho da cache, do tamanho do bloco, da associatividade e da largura de banda da memória, além de considerar variáveis como número de blocos, política de escrita, algoritmo de substituição e associatividade. A pesquisa avalia as taxas de acerto, as médias de acesso e as operações de leitura e escrita na Memória Principal, visando entender o funcionamento do sistema computacional.

1. INTRODUÇÃO

Um dos aspectos mais desafiadores do projeto de um computador em toda a história tem sido oferecer um sistema de memória capaz de fornecer operandos ao processador à velocidade em que ele pode processá-los. Um modo de atacar esse problema é providenciar caches [1]. A Memória Cache desempenha um papel fundamental de diminuir a latência de acesso e aumentar a taxa de acertos, impactando diretamente o desempenho do sistema computacional. O uso de Memória Cache visa obter uma velocidade de acesso à memória próxima da velocidade das memórias mais rápidas e, ao mesmo tempo, disponibilizar no sistema uma memória de grande capacidade, a um custo equivalente ao das memórias de semicondutor mais baratas [2].

A partir da contextualização e problemática apresentadas, o presente estudo tem como objetivo apresentar os resultados obtidos através da simulação de uma Memória Cache associativa por conjunto com arquitetura configurável. A pesquisa permite várias análises, tais como: o impacto do tamanho da cache, o impacto do tamanho do bloco, o impacto da associatividade, largura de banda da memória e uma avaliação global. A partir da mudança dos parâmetros – número de blocos, política de escrita, algoritmo de substituição e associatividade – é possível observar as variações nas taxas de acerto, nas taxas médias de acesso e, por fim, nas leituras e escritas realizadas na Memória Principal.

2. MATERIAL E MÉTODOS

A fim de simular o funcionamento de uma Memória Cache associativa por conjunto, foi desenvolvido um algoritmo, escrito na linguagem: C/C++. Neste algoritmo foram declarados 3 *structs* como objetivo de representar: a Memória Cache, os conjuntos e os blocos (linhas). Na *Struct* chamada “Cache” foram declarados os parâmetros de configuração da Memória Cache utilizando o tipo *integer*:

- **numConjuntos**: representa o número total de conjuntos na Memória Cache;

- **linhas_por_conjuntos**: representa o número total de linhas na cache;
- **tamanhoLinha**: representa o tamanho da linha na cache;
- **hit_time**: indica o tempo utilizado ao considerar um *cache-hit*;
- **politica_escrita**: utilizado para indicar qual política de escrita está sendo utilizada, sendo 0 representando a política *Write-through* e 1 representando a política *Write-back*.
- **politica_substituicao**: indica qual a política de substituição sendo utilizada, sendo 0 para representar a política LFU, 1 para representar a LRU e 2 para representar a política Randômica.

Também foi declarado um ponteiro referenciando um vetor de *Structs* “CacheConjunto”, este por sua vez contendo um ponteiro referenciando um vetor de *Structs* “CacheLinha”.

Na *Struct* “CacheLinha”, foram declarados os parâmetros referentes ao bloco da Memória Cache:

- **tag**: em formato *unsigned integer*, para representar a tag correspondente a cada linha da memória;
- **valido**: *integer* utilizado para representar se a linha contém ou não dados.
- **dirty**: *integer* representando o *dirty bit*.
- **ultima_vez_usado**: contador, utilizado para indicar quando a linha foi utilizada.
- **frequencia**: contador, utilizado para identificar a frequência com que cada linha do conjunto é utilizada.

No início do algoritmo, são declarados os parâmetros principais para a simulação. Com esses parâmetros definidos, a função *inicializa_cache* é chamada. Esta função realiza a inicialização e alocação das *Struct* anteriormente citadas, de acordo com os parâmetros de entrada previamente declarados. Após a alocação e inicialização da estrutura, algumas variáveis utilizadas para guardar os valores de saída do algoritmo são inicializadas com o seu valor em 0. E então, através da utilização de um laço de repetição, é realizada a leitura linha a linha do arquivo de entrada que contém os endereços em formato hexadecimal e a operação a ser simulada, no formato: “0020a9a0 R”, no qual os 8 primeiros caracteres representam o endereço e o último caractere a operação, sendo R a operação de leitura (*read*) e W a operação

de escrita (*write*). No início da leitura da linha do arquivo o endereço e operação são separadas e salvas em variáveis distintas: o endereço em formato *unsigned int* e operação em formato *char*. Em seguida, o contador que representa o total de endereços é incrementado e, de acordo com a variável indicando a operação, o contador de total de escritas ou total de leituras é incrementado.

As operações de separação de dígitos são determinadas e salvas em variáveis a parte do endereço que representa o conjunto na Memória Cache e a parte que representa a *tag* da linha. As operações utilizadas estão descritas abaixo:

$$conjunto = \frac{endereco}{tamanhoLinha} \% numConjuntos$$

$$tag = \frac{endereco}{tamanhoLinha \times numConjuntos}$$

Nelas, o *conjunto* e a *tag* são as variáveis utilizadas para salvar os seus respectivos valores; o *endereço*, por sua vez, é a variável contendo o endereço lido no arquivo de entrada; *cache.tamanhoLinha* é o parâmetro indicando o tamanho da linha na simulação; e *cache.numConjuntos* indica o número de conjuntos utilizado.

Com esses valores definidos, é utilizado um laço para percorrer o conjunto a fim de encontrar a linha que corresponde à *tag* do endereço de entrada. Quando a linha é encontrada, é realizada uma verificação se o parâmetro *valido* é verdadeiro ou falso. Caso seja verdadeiro, indica que nessa linha da Memória Cache já foram carregados dados, o que representa um acerto (*memory hit*). Nesse caso, a variável *hit* é definido com o valor 1 e os contadores da linha, *frequencia* e *ultima_vez_usado* são incrementados. Se a operação for de escrita e a política escrita adotada for a *Write-back*, o indicador *dirty* é definido como 1 para a linha encontrada. Após isso, o contador de *hits* da operação é incrementado. No caso de uma operação de leitura e uma política de escrita *Write-through*, o contador de escritas na memória principal é incrementado. Posteriormente, o laço de repetição passará para a próxima linha do arquivo de entrada.

Caso todas as linhas do conjunto sejam percorridas e nenhuma delas contenha dados, acontece um *memory miss*. De acordo com a política de substituição utilizada, a linha recebe os dados da operação com o propósito de armazená-los na cache.

No caso da política de LFU, as linhas do bloco são percorridas a fim de encontrar aquela que possui o contador *ultima_vez_usado* com o menor valor e, ao mesmo tempo, é salvo na variável *maior_ultima_vez_usado* o maior valor encontrado para *ultima_vez_usado*.

Já para a política LRU, a linha buscada para substituição será a linha que contém o contador *frequencia* com o menor valor.

Na política Randômica, a linha é selecionada aleatoriamente.

Uma vez encontrada a linha que receberá os dados da operação atual, verifica-se se o indicador de *dirty bit* está definido como 1. Em caso afirmativo, isso indica que já existem dados nela. Dessa forma, realiza-se o incremento do contador de total de escritas na Memória Principal, representando a escrita desse bloco na MP.

Após isso, o contador de leituras na Memória Principal é incrementado, simulando a busca da informação a ser guardada na cache na MP. Em seguida, os parâmetros da linha são atualizados: o indicador *valido* é definido como 1, indicando que a linha possui dados; *ultima_vez_usado*, recebe o incremento da variável *maior_ultima_vez_usado*; o indicador *frequencia* é incrementado; o indicador de *tag* recebe a rótulo do endereço; e o indicador *dirty* é zerado.

Depois de simulado o salvamento do dado na cache, caso a operação seja de escrita e a política seja *Write-back*, o indicador *dirty* da linha é configurado para 1. Já no caso de ser *Write-through*, é incrementado o contador de escritas na MP.

Dessa forma, após percorrer todas as linhas do arquivo de entrada, e no caso de a política de escrita ser *Write-back*, ocorre o descarregamento de todas as linhas que foram modificadas. Para isso, todas as linhas da cache são percorridas e o contador de escritas na MP é incrementado para todas as linhas que possuem o indicador de *dirty bit* igual a 1.

Então, os parâmetros de saída são calculados com base nos contadores que foram atualizados durante a execução dos passos anteriores, eles são salvos linha a linha no arquivo de saída.

As análises utilizando o algoritmo de simulação da Memória Cache foram realizadas em 5 seções, a fim de estudar diferentes parâmetros e sua influência no processo de execução e funcionamento da cache, essas seções são:

- Impacto do Tamanho da Cache;
- Impacto do Tamanho do Bloco;
- Impacto da Associatividade;
- Impacto da Política de Substituição;
- Largura de Banda da Memória;
- Avaliação Global;

Para a análise de Impacto do Tamanho na Cache, os experimentos foram realizados utilizando blocos com 128 bytes, política de escrita *Write-through*, política de substituição LRU e associatividade de 4 blocos. Foram realizadas 7 simulações, variando a quantidade de blocos, iniciando em 16 e terminando em 1024, simulando consequentemente uma Memória Cache de maior tamanho em cada experimento. O parâmetro de saída observado foi a variação da taxa de acerto em função do aumento do tamanho da Memória Cache.

Ao analisar o Impacto do Tamanho do Bloco na Cache, utilizou-se um tamanho da Memória Cache de 8 Kbytes, associatividade de 2 blocos, política de escrita *Write-through* e política de substituição LRU. Foram realizadas ao todo 11 simulações, variando o tamanho do bloco de 4 a 4Kbytes em potências de 2, e consequentemente variando o número de

blocos de 2048 até 2, também em potência de 2. Nessa análise, observou-se a variação da taxa de acerto em função do aumento do tamanho do bloco.

Já para o Impacto da Associatividade, também se utilizou um tamanho da memória de 8Kbytes, tamanho de bloco de 128 bytes, política de escrita *Write-back* e política de substituição LRU. A associatividade foi variada entre 1 e 64 em potências de 2, totalizando 8 simulações. Observou-se a alteração na taxa de acerto de acordo com o aumento da associatividade.

Para a análise do impacto da Política de Substituição, foi utilizado tamanho de bloco de 128 bytes, política de escrita *Write-through*, associatividade de 4 blocos. O número de bloco foi variado entre 16 e 1024 em potências de 2. As simulações foram realizadas para cada uma das políticas de substituição: LRU, LFU e randômica. O objetivo foi analisar como as diferentes políticas de substituição afetam a taxa de acerto conforme o tamanho da cache aumenta.

E por fim, para a Análise de largura de banda da memória, foram simuladas memórias cache de 8 e 16KB, com tamanhos de blocos com 64 e 128 bytes, associatividade de 2 e 4 e política de substituição LRU. Este experimento foi realizado para as duas políticas de escrita: *Write-back* e *Write-through*, totalizando 16 simulações (8 para cada política). O objetivo foi analisar a quantidade de acesso à MP.

3. RESULTADOS

Impacto do Tamanho da Cache

O experimento foi realizado com blocos de 128 bytes, uma política de escrita *Write-through*, uma política de substituição LRU e uma associatividade de 4 blocos. O número de blocos foi o único parâmetro alterado: iniciou-se com 16 blocos e, em potência de 2, os eles foram aumentando até o limite de 1024.

Observou-se (Figura 1) que a taxa de acerto cresce conforme o tamanho da cache aumenta. No entanto, é importante salientar que a taxa de acerto, após um determinado tamanho da cache, não sofre mais alterações evidentes — sua funcionalidade permanece praticamente estática. Dessa forma, é possível constatar que o tamanho da Memória Cache deve ser suficientemente pequeno para que o custo total médio por bit seja próximo do custo por bit da Memória Principal e deve ser suficientemente grande para que o tempo médio de acesso à memória seja próximo ao tempo de acesso da Memória Cache. Quanto maior é a Memória Cache, maior é o número de portas envolvidas no endereçamento e, como resultado, se tornam mais lentas que as pequenas. Além disso, o seu tamanho também é limitado pela dimensão da pastilha e da placa de circuito. Dessa forma, conforme os estudos sugerem, memórias cache com tamanho entre 1K e 512K palavras são mais eficientes [2].

Constatou-se também que a curva da taxa de acerto em função do tamanho da cache, pode ser dividida em 3 fases: fase de crescimento rápido, fase de crescimento moderado e fase de saturação.

- **Fase crescimento rápido:** Quando a cache é pequena, aumentar o seu tamanho resulta em um aumento

significativo na taxa de hits. Isso acontece pois conforme o tamanho da cache aumenta, mais dados são armazenados e menor vai ser a necessidade de substituir alguma das linhas que já estão em uso;

- **Fase de crescimento moderado:** Conforme o tamanho da cache continua crescendo, o aumento na taxa de acerto tende a diminuir, isso acontece, pois, a maioria dos dados que são mais frequentemente acessados, já estão sendo armazenados na cache.
- **Fase de saturação ou estabilização:** Existe um ponto em que aumentar a cache não terá mais nenhum efeito na taxa de acerto, pois o tamanho da cache já é suficientemente grande para comportar todos os dados de acesso, apenas os acessos únicos geram um *miss*.

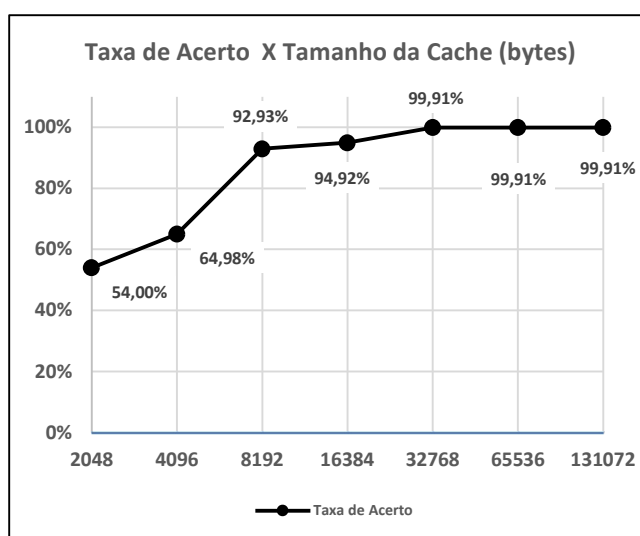


Figura 1. Taxa de acerto em relação ao tamanho da cache

Impacto do Tamanho do Bloco

Para o segundo experimento, o tamanho do bloco foi variado de 4 a 4Kb bytes, em potência de 2. A Memória Cache, por sua vez, possui 8 Kbytes com uma associatividade de 2 blocos. As políticas utilizadas foram, respectivamente: *Write-through* em LRU.

Observou-se (Figura 2) que a taxa de acerto diminui conforme o tamanho do bloco aumenta. Esse fator está intrinsecamente associado ao princípio de localidade de referência, já que quando um bloco de dados é trazido para a cache para satisfazer uma dada referência à memória, é provável que futuras referências sejam feitas para outras palavras desse bloco [2]. Esse processo tende a ramificar-se em dois princípios: localidade temporal e localidade espacial. Este é responsável pela proximidade dos dados, já que eles tendem a ser acessados juntos, diminuindo, assim, a frequência de acesso a memória. Aquele, por sua vez, atenta-se ao acesso mais recente, ou seja, dados já acessados tendem a ser acessados novamente em um curto período. Isso ocorre, por exemplo, com localizações de memória próximas ao topo da pilha, ou com instruções dentro de um laço. Muitos algoritmos de substituição de cache também exploram a localidade temporal, descartando as entradas que não foram

acessadas recentemente [1].

À medida que se amplia o tamanho do bloco, a taxa de acerto na Memória Cache inicialmente aumenta, em razão do princípio de localidade de referências. Com um tamanho de bloco maior, mais dados úteis são trazidos para a Memória Cache. Entretanto, a taxa de acerto tende a diminuir se o tamanho do bloco se torna tão grande que a probabilidade de utilizar os dados buscados recentemente se torna menor do que a probabilidade de reutilizar os dados que foram substituídos. Dois efeitos específicos devem ser considerados:

- Para uma dada capacidade, o uso de blocos maiores reduz o número de blocos contidos na Memória Cache. Como cada novo bloco trazido para a Memória Cache é escrito sobre um bloco anteriormente armazenado, um pequeno número de blocos faz com que os dados sejam sobrescritos tão logo sejam buscados.
- Escolhendo-se blocos de maior tamanho, cada palavra adicional está mais distante da palavra usada e, portanto, tem menor probabilidade de ser utilizada em um futuro próximo [2].

A relação entre o tamanho do bloco e a taxa de acerto é bastante complexa. Nenhum valor *ótimo* definitivo ainda foi determinado. Um tamanho de bloco de duas a oito palavras parece estar razoavelmente próximo ao ótimo [2].

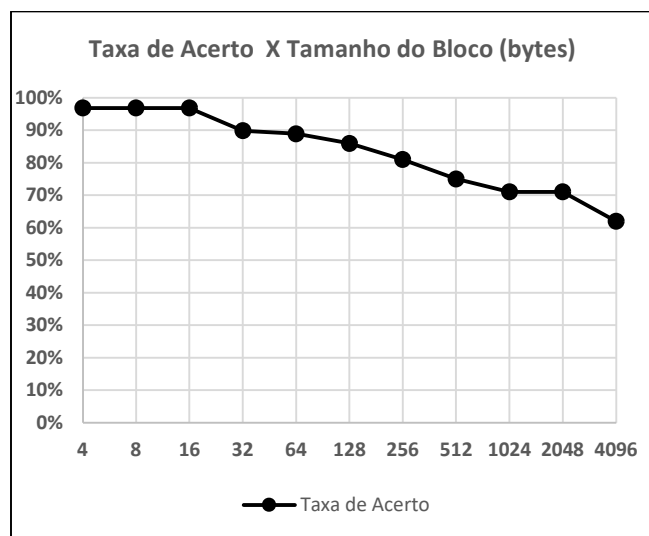


Figura 2. Taxa de acerto em relação ao tamanho do bloco

Impacto da Associatividade

Para o terceiro experimento, o parâmetro variado foi a associatividade: entre 1 e 64, em potência de 2, para uma cache de 8Kbytes.

Observou-se (Figura 3) que caches com alto grau de associatividade não melhoram muito o desempenho em relação a caches de baixo grau de associatividade sob a maioria das circunstâncias e, em alguns casos, até funcionam pior — nos testes, quando a associatividade estava em 8, por exemplo, a taxa de acerto teve uma queda considerável, vide gráfico abaixo. Por essas razões, a associatividade de conjunto além de quatro vias é relativamente incomum [1].

No caso extremo, ou seja, $v = m$ (conjuntos = número de linhas na Memória Cache) e $k = 1$ (linhas = 1), a técnica de mapeamento associativo por conjuntos se reduz ao mapeamento direto. Para $v = 1$ (conjuntos = 1) e $k = m$ (linhas = número de conjuntos na Memória Cache), ela se reduz ao mapeamento associativo. A organização de mapeamento associativo por conjunto mais comum é a que usa duas linhas por conjunto ($v = m/2$, $k = 2$). Sua taxa de acertos é significativamente maior no que no caso do mapeamento direto. Um mapeamento associativo por conjunto de quatro linhas ($v = m/4$, $k = 4$) apresenta uma pequena melhoria a custo adicional relativamente pequeno [2].

Esse comportamento crescente na taxa de acerto de acordo com o aumento da associatividade pode ser explicado através da redução de conflitos na cache. Com mais blocos por conjunto, é menos provável que os blocos frequentemente acessados sejam substituídos por outros. Porém, em determinado ponto, o aumento no número de blocos por conjunto já não surte mais efeito no aumento da taxa de acerto, pois o tamanho da cache é grande o suficiente para que não ocorram conflitos de endereçamento.

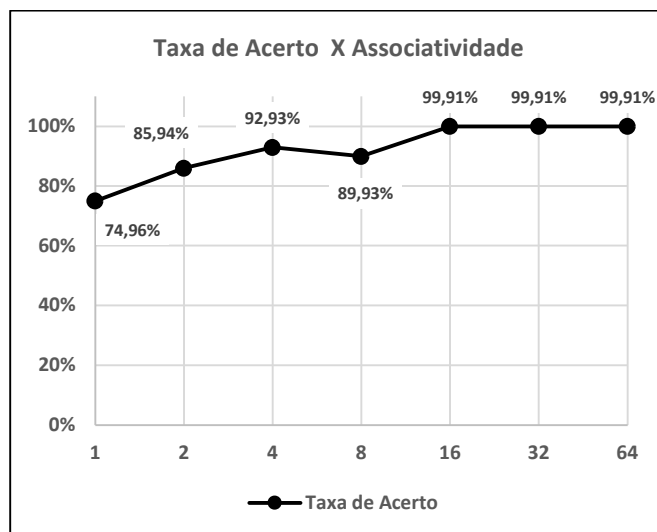


Figura 3. Taxa de acerto em relação a associatividade

Impacto da Política de Substituição

Para o quarto experimento, os parâmetros variados foram o número de blocos da cache e a política de substituição: LFU, LRU e aleatória. Iniciou-se com 16 blocos e, em potência de 2, os eles foram aumentando até o limite de 1024.

Observou-se (Figura 4) que a política de substituição Randômica apresentou uma taxa de acerto maior em relação as outras, LRU e LFU — no entanto, todos os testes apresentaram valores muito próximos. Fato este que está em desacordo com as referências encontradas na literatura que embasam este relatório: o algoritmo mais eficaz é, provavelmente, o baseado na política de substituir o bloco menos recentemente usado. Supondo que as posições de memória mais recentes usadas tenham maior probabilidade de ser referenciadas em um futuro próximo, o algoritmo de LRU deve fornecer a melhor taxa de acerto. Já o LFU apresenta uma

taxa levemente inferior ao LRU. O Randômico, por sua vez, a partir de estudos baseados em simulação, mostram que a substituição aleatória apresenta um desempenho apenas levemente inferior ao de um algoritmo baseado no histórico de uso das linhas [2].

O comportamento demonstrado na simulação gerou uma imprecisão nos resultados obtidos. Por conta disso, na tentativa de validar o código do simulador, foram gerados, aleatoriamente, novos dados de acesso a memória com uma amostragem maior: 200.000 linhas. O resultado desse novo teste corroborou com as diferenças de comportamento entre as políticas de substituição encontradas na literatura: a política LRU com taxa de acertos um pouco maior que a política LFU e a política Randômica com uma taxa de acerto levemente menor. Esses dados podem ser vistos na Tabela 1.

Tendo em vista esse resultado, constatou-se que o que pode ter causado as diferenças em relação à literatura de referência foi a baixa amostragem nos dados de entrada ou alguma especificidade quanto aos endereços contidos neles.

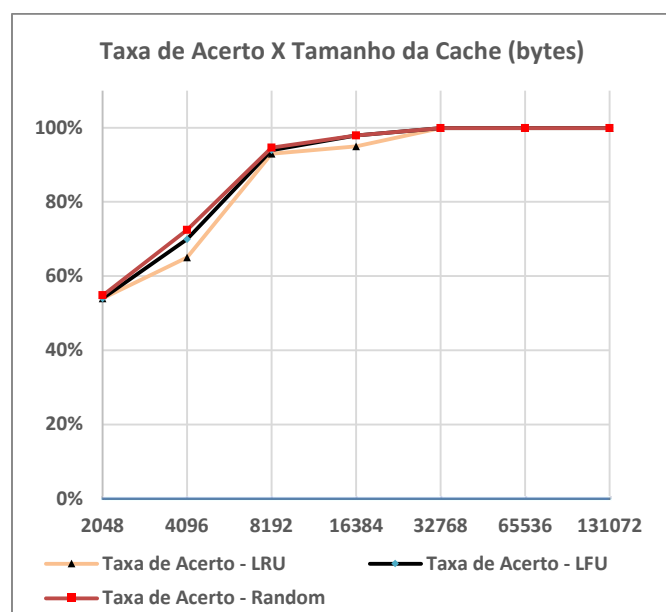


Figura 4. Taxa de acerto em relação ao tamanho da cache

Número de Blocos	Tamanho da Cache	LRU	LFU	RANDOM
		Taxa de Acerto	Taxa de Acerto	Taxa de Acerto
1024	131072	50,46%	50,43%	50,34%
2048	262144	81,01%	80,90%	80,45%
4096	524288	95,89%	95,88%	95,50%
8162	1048576	95,89%	95,88%	95,50%

Tabela 1. Resultados de taxa de acerto em relação ao tamanho da cache, para simulação com endereços gerados aleatoriamente.

Largura de Banda da Memória

Para a quinta análise, foram simuladas memórias caches de 16 e 8 Kb, utilizando associatividade de 2 e 4 blocos, e tamanhos de blocos de 64 e 128 bytes, para as políticas de escrita *Write-through* e *Write-back*, totalizando 8 simulações para cada política. Os resultados das simulações se encontram nas tabelas 2 e 3.

Conforme observado nos resultados, pode-se inferir que a política de escrita *Write-back* resulta em um total de acessos à memória menor do que a política *Write-through*, principalmente em relação ao número de escritas na memória, visto que o número de leituras foi idêntico para as duas políticas.

Na política *Write-back*, os dados são gravados na Memória Principal apenas quando ocorre a substituição de uma das linhas do bloco e ao finalizar simulação, quando todas as linhas marcadas como *dirty* são escritas na MP. De acordo com [2], a escrita de volta (*write-back*), visa minimizar o número de operações de escrita na memória. Nessa técnica, as escritas são feitas apenas na memória cache. Quando é feita uma atualização, é atribuído o valor 1 a um bit de ATUALIZAÇÃO, associado à linha atualizada na memória cache. Quando o bloco vai ser substituído, ele apenas é escrito de volta na memória principal se seu bit de ATUALIZAÇÃO tiver valor 1.

Já na política *Write-through*, as escritas na MP ocorrem conforme as operações de escrita são realizadas, numa proporção de 1:1, ou seja, para cada operação de escrita dos dados de entrada, é realizada uma escrita na MP. Conforme [2], a técnica de atualização mais simples é denominada escrita direta (*write-through*). Nessa técnica, todas as operações de escrita são feitas tanto na memória cache quanto na memória principal, assegurando assim que a memória principal esteja sempre com os dados válidos. [...] A principal desvantagem dessa técnica é que ela gera um tráfego de memória considerável, podendo criar um gargalo no sistema.

A experiência mostra que as operações de escrita constituem aproximadamente 15% das referências à memória [2]. Nas simulações realizadas as operações de escritas equivalem as 12% do total de operações: 6144 de um total de 51200 operações.

Write-Back					
Tam. da Memória (Kb)	Tam. do Bloco (bytes)	Assoc.	Leituras na MP	Escritas na MP	Total de acessos a MP
8	64	2	5681	1026	6707
8	64	4	3126	515	3641
8	128	2	7198	1026	8224
8	128	4	3621	515	4136
16	64	2	3126	1026	4152
16	64	4	2615	515	3130
16	128	2	4643	1026	5669
16	128	4	2599	515	3114
Médias			4076,125	770,5	4846,625

Tabela 2. Médias de números de acessos na Memória Principal utilizando política *Write-back*.

Write-Through					
Tam. da Memória (Kb)	Tam. do Bloco (bytes)	Assoc.	Leituras na MP	Escritas na MP	Total de acessos a MP
8	64	2	5681	6144	6707
8	64	4	3126	6144	3641
8	128	2	7198	6144	8224
8	128	4	3621	6144	4136
16	64	2	3126	6144	4152
16	64	4	2615	6144	3130
16	128	2	4643	6144	5669
16	128	4	2599	6144	3114
Médias			4076,125	6144	10220,125

Tabela 3. Médias de números de acessos na Memória Principal utilizando política *Write - Through*

4. CONCLUSÕES

Com base nos resultados das simulações realizadas, nas informações encontradas na literatura e nas análises discutidas, é possível estabelecer os parâmetros que representem o “melhor projeto de cache”. Os parâmetros que serão determinados são: tamanho total da cache, tamanho do bloco, associatividade, política de substituição e política de escrita.

Para o tamanho total da cache, com base nos resultados da análise 1, o tamanho ideal estaria entre 8Kb e 32Kb. Memórias acima de 32Kb apresentam o mesmo desempenho de taxa de acerto, enquanto memórias abaixo de 8Kb tem uma taxa de acerto muito inferior. Dentro dessa faixa considerada como ideal, para escolher um tamanho específico, deve-se levar em conta a disponibilidade de recursos. Uma memória de 32Kb, por exemplo, apresentará o melhor resultado em relação a taxa de acerto, 99,91%. No entanto, ela se torna muito mais cara que uma memória de 8Kb, com taxa de acerto de 92,93% — um ganho de apenas 6,98%, apesar de ter um tamanho quatro vezes maior.

Para a escolha do tamanho do bloco, de acordo com o que está sendo apresentado na análise 2, o ideal se dá em torno de 16 bytes. Este tamanho apresenta a mesma taxa de acerto do que tamanhos menores, ao mesmo tempo em que reduz a quantidade total de blocos. Isso maximiza a taxa de acertos, ao mesmo tempo que minimiza a frequência de acesso à Memória Principal, uma vez que mais dados serão trazidos de uma vez para dentro da Memória Cache — além de diminuir a quantidade total de blocos em relação a um bloco de menor tamanho.

Quanto ao parâmetro associatividade, observando a análise 3, dois valores apresentam-se como opções para compor a Memória Cache ideal: 4 ou 16. Com uma associatividade de 4, foi obtido uma taxa de acerto de 92,93%, enquanto com associatividade de 16, obteve-se uma taxa de acerto máxima de 99,91%.

Assim como na escolha do tamanho máximo da cache, é preciso observar a disponibilidade de recursos ao optar por uma associatividade maior, já que envolve mais custos.

Para a política de substituição, nas simulações foram obtidos resultados muito próximos para todos os tamanhos de cachê analisados. Dessa forma, deve-se analisar a influência da complexidade das políticas de substituição em relação ao custo final de produção, sendo que, de forma geral, a política LRU apresentam resultados levemente melhores, conforme visto em [2].

Para determinar a política de escrita, observa-se que, em geral, a política *Write-back* resulta em menos acessos à Memória Principal e, por consequência, gera um menor tráfego de dados, tornando a memória mais eficiente.

Desse modo, os parâmetros escolhidos podem ser vistos a seguir (Tabela 4):

Parâmetro	Valor Escolhido
Tamanho da Memória Cache	8Kb a 32Kb
Tamanho do bloco	16 bytes
Associatividade	4 ou 16
Política de substituição	LRU
Política de escrita	Write-back

Tabela 4. Valores escolhidos para compor o projeto de Memória Cache “ideal”.

Tendo em vista que o código utilizado para realizar as simulações permite a seleção dos parâmetros de associatividade e número de linhas, é possível simular as demais formas de mapeamento, conforme visto em [2]. Por exemplo, para uma associatividade de 1, ou seja, 1 linha por conjunto, a técnica de mapeamento associativo por conjuntos se reduz ao mapeamento direto. Já para associatividade igual ao número de conjuntos na Memória Cache, a técnica se reduz a mapeamento associativo.

Durante o processo de desenvolvimento do simulador de Memória Cache totalmente configurável, foram identificadas algumas dificuldades, dentre elas, destacam-se: a validação dos dados obtidos durante a simulação; e a validação do processo de desenvolvimento da simulação das políticas de escrita e das políticas de substituição.

5. BIBLIOGRAFIA

- [1] TANEMBAUM, Andrew S. **Organização estruturada de computadores**. 5. ed. São Paulo: Person Hall, 2007.
- [2] STALLINGS, William. **Arquitetura e organização de computadores: projeto para o desempenho**. 5. ed. São Paulo: Pearson Hall, 2002.

LINKS DE ACESSO AO CÓDIGO:

- <https://github.com/gabrielmazza/SimuladorMemoriaCache>
- <https://drive.google.com/file/d/1AGoRCUMGdd1xdeY2YWV5ybQb98CUV5Jq/view?usp=sharing>

ANEXO A**ANÁLISE 1****Parâmetros da Simulação**

Parâmetro	Valor
Tamanho do Bloco	128 bytes
Política de Escrita	Write-through
Algoritmo de Substituição	LRU
Associatividade	4 blocos

Resultados da Simulação

Número de Blocos	Taxa de Acerto	Tempo médio de Acesso (ns)	Leituras na MP	Escritas na MP
16	54,00%	37,6	23552	6144
32	64,98%	31,015	17929	6144
64	92,93%	14,2434	3621	6144
128	94,92%	13,0457	2599	6144
256	99,91%	10,0656	44	6144
512	99,91%	10,0656	44	6144
1024	99,91%	10,0656	44	6144

ANÁLISE 2

Parâmetros da Simulação

Parâmetro	Valor
Tamanho da Cache	8 Kbytes
Política de Escrita	Write-through
Algoritmo de Substituição	LRU
Associatividade	2 blocos

Resultados da Simulação

Número de Blocos	Tamanho do Bloco	Taxa de Acerto	Tempo médio de Acesso (ns)	Leituras na MP	Escritas na MP
2048	4	96,82%	11,9066	1627	6144
1024	8	96,82%	11,9066	1627	6144
512	16	96,82%	11,9055	1626	6144
256	32	89,84%	16,0961	5202	6144
128	64	88,90%	16,6574	5681	6144
64	128	85,94%	18,4352	7198	6144
32	256	80,97%	21,4176	9743	6144
16	512	74,99%	25,0047	12804	6144
8	1024	71,00%	27,4	14848	6144
4	2048	71,00%	27,4012	14849	6144
2	4096	62,00%	32,8	19456	6144

ANÁLISE 3**Parâmetros da Simulação**

Parâmetro	Valor
Tamanho da Cache	8 Kbytes
Tamanho do Bloco	128 bytes
Número de Linhas	64
Política de Escrita	Write-through
Algoritmo de Substituição	LRU

Resultados da Simulação

Associatividade	Taxa de Acerto	Tempo médio de Acesso (ns)	Leituras na MP	Escritas na MP
1	74,96%	25,0258	12822	2048
2	85,94%	18,4352	7198	1026
4	92,93%	14,2434	3621	515
8	89,93%	16,0398	5154	1026
16	99,91%	10,0516	44	4
32	99,91%	10,0516	44	4
64	99,91%	10,0516	44	4

ANÁLISE 4

Parâmetros da Simulação

Parâmetro	Valor
Tamanho do Bloco	128 bytes
Política de Escrita	Write- back
Algoritmo de Substituição	LRU
Associatividade	4 blocos

Resultados da Simulação

LRU:

Número de Blocos	Tamanho da Cache	Tempo médio de Acesso (ns)	Leituras na MP	Escritas na MP	Taxa de Acerto
16	2048	38	23552	6144	54,00%
32	4096	31	17929	6144	64,98%
64	8192	14,2434	3621	6144	92,93%
128	16384	13,0457	2599	6144	94,92%
256	32768	10,0516	44	6144	99,91%
512	65536	10,0516	44	6144	99,91%
1024	131072	10,0516	44	6144	99,91%

LFU:

Número de Blocos	Tamanho da Cache	Tempo médio de Acesso (ns)	Leituras na MP	Escritas na MP	Taxa de Acerto
16	2048	38	23553	6144	54,00%
32	4096	28	15375	6144	69,97%
64	8192	13,6445	3110	6144	93,93%
128	16384	11,2492	1066	6144	97,92%
256	32768	10,0516	44	6144	99,91%
512	65536	10,0516	44	6144	99,91%
1024	131072	10,0516	44	6144	99,91%

RANDOM:

Número de Blocos	Tamanho da Cache	Tempo médio de Acesso (ns)	Leituras na MP	Escritas na MP	Taxa de Acerto
16	2048	37	23131	6144	54,82%
32	4096	27	14095	6144	72,47%
64	8192	13,2309	2757	6144	94,62%
128	16384	11,2668	1081	6144	97,89%
256	32768	10,0609	52	6144	99,90%
512	65536	10,0551	47	6144	99,91%
1024	131072	10,0539	46	6144	99,91%

ANÁLISE 5

Parâmetros da Simulação

Parâmetro	Valor
Algoritmo de Substituição	LRU

Resultados da Simulação

Write – Back:

Tam. da Memória (Kb)	Tam. do Bloco (bytes)	Assoc.	Leituras na MP	Escritas na MP	Total de acessos a MP
8	64	2	5681	1026	6707
8	64	4	3126	515	3641
8	128	2	7198	1026	8224
8	128	4	3621	515	4136
16	64	2	3126	1026	4152
16	64	4	2615	515	3130
16	128	2	4643	1026	5669
16	128	4	2599	515	3114
Médias			4076,125	770,5	4846,625

Write – Through:

Tam. da Memória (Kb)	Tam. do Bloco (bytes)	Assoc.	Leituras na MP	Escritas na MP	Total de acessos a MP
8	64	2	5681	6144	11825
8	64	4	3126	6144	9270
8	128	2	7198	6144	13342
8	128	4	3621	6144	9765
16	64	2	3126	6144	9270
16	64	4	2615	6144	8759
16	128	2	4643	6144	10787
16	128	4	2599	6144	8743
Médias			4076,125	6144	10220,125